SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1703**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E.Ragul

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu
Ii. If a patient has Fever AND Rash → Possible Measles
Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia
Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

ANSWER:

This case study demonstrates **Knowledge Representation and Reasoning using Propositional Logic, Forward Chaining, and Backward Chaining**.

(a) Represent the Rules and Facts using Propositional Logic
Step 1: Define the symbols

Symbol	Meaning
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Flu	Patient has Possible Flu
M	Patient has Possible Measles
P	Patient has Possible Pneumonia

Here, **AND** is represented by **∧**, and **IF...THEN** is represented by →.

Step 2: Represent the rules

Rule I:

If a patient has Fever AND Cough → Possible Flu.

$F \land C \rightarrow Flu$

Rule II:

If a patient has Fever AND Rash → Possible Measles.

$F \land R \rightarrow M$

Rule III:

If a patient has Cough AND Breathlessness → Possible Pneumonia.

$C \wedge B \rightarrow P$

Step 3: Represent Patient A's facts

Patient A has:

- Fever = True
- Cough = True
- Breathlessness = True

Therefore:

F C B

There is **no fact stating that Patient A has Rash**, so we cannot assume:

R

(b) Apply Forward Chaining
What is Forward Chaining?

Forward Chaining starts with the **known facts** and repeatedly applies rules whose conditions are satisfied to derive new facts.

Initial Knowledge Base

Known facts:

F, C, B

That means:

- Fever = True
- Cough = True
- Breathlessness = True

Step 1: Check Rule I

Rule I:

$F \wedge C \rightarrow Flu$

We know:

$F = True$

and

$C = True$

Therefore, both conditions are satisfied

Derive:
Flu

Conclusion: Patient A has **Possible Flu**.

Step 2: Check Rule II

Rule II:

$F \wedge R \rightarrow M$

We know:

$F = True$

But there is **no fact that R = True**.

Therefore, the rule **cannot be fired**.

Result:
M cannot be derived

So, we cannot conclude that Patient A has Possible Measles.

Step 3: Check Rule III

Rule III:

$C \wedge B \rightarrow P$

We know:

$C = True$

and

$B = True$

Therefore, both conditions are satisfied.

Derive:
P

Conclusion: Patient A has **Possible Pneumonia**.

Forward Chaining Inference Table

Step	Rule	Conditions	New Fact
Initial	—	F, C, B are known	Fever, Cough, Breathlessness
1	Rule I	F ∧ C	**Flu**
2	Rule II	F ∧ R; R unavailable	Nothing
3	Rule III	C ∧ B	**Pneumonia**

Final conclusions

From the given knowledge base:

Flu

and

Pneumonia

can be derived.

Measles cannot be derived because Rash is not given as a fact.

In a real medical system, these are rule-based **possible diagnoses**, not confirmed clinical diagnoses.

(c) Backward Chaining for "Patient A has Pneumonia"
What is Backward Chaining?

Backward Chaining starts with the **goal** and works backward to determine which rules and facts can prove that goal.

Goal:
P

That means:

Goal: Patient A has Possible Pneumonia.

Step 1: Find a rule that concludes Pneumonia

Look at Rule III:

$C \wedge B \rightarrow P$

This rule can produce the goal **P**.

Therefore, to prove **P**, we need to prove:

$C \wedge B$

So the goal is reduced into two sub-goals:

1. Cough
2. Breathlessness

Step 2: Verify Cough

We already have the fact:

C
C=True

The first sub-goal is satisfied.

Step 3: Verify Breathlessness

We already have the fact:

B

Therefore:

B=True

The second sub-goal is also satisfied.

Goal Reduction Tree

You can draw the tree in your exam like this:

```
                GOAL
     Patient A has Pneumonia (P)
                   |
             Rule III: C ∧ B → P
                   |
            C ∧ B required
              /         \
             /           \
Cough (C)      Breathlessness (B)
          |                |
  Given Fact          Given Fact
  C = True            B = True
          \                /
           \              /
            \            /
                   |
              GOAL SATISFIED
                   |
          Patient A has Pneumonia
```

In symbolic form:

P

C∧B

Given:

C=True

and

B=True

C∧B=True

Hence:

P=True

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. $\forall x$: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. $\forall x$: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. $\forall x\ \forall y$: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

ANSWER:

This case study demonstrates First-Order Logic (FOL), Unification, Resolution, CNF conversion, and logical reasoning for an autonomous traffic management system.

(a) Apply Unification to Rule 1
Given Rule 1
$\forall x$: Vehicle(x)∧EmergencyType(x)→ClearPath(x)
Given facts:
Vehicle(Ambulance) EmergencyType(Ambulance)
We need to match the variable x in Rule 1 with the given facts.

Step 1: Match Vehicle(x)
Rule 1 contains:
Vehicle(x)
Fact:
Vehicle(Ambulance)
To make them identical:
x=Ambulance
Therefore, the substitution is:
$\theta 1$={x/Ambulance}

Step 2: Match EmergencyType(x)
Rule 1 contains:
EmergencyType(x)
Using the substitution:
x=Ambulance
we get:
EmergencyType(Ambulance)
This exactly matches the given fact.
Therefore:
$\theta 2$={x/Ambulance}

Step 3: Final MGU
Both conditions are satisfied using the same substitution:
θ={x/Ambulance}
This is the Most General Unifier (MGU).
After applying the substitution to Rule 1:
Vehicle(Ambulance)∧EmergencyType(Ambulance)→ClearPath(Ambulance)
Since both facts are true:
ClearPath(Ambulance)

Unification Table

Step	Expression	Fact	Substitution
1	Vehicle(x)	Vehicle(Ambulance)	x/Ambulance
2	EmergencyType(x)	EmergencyType(Ambulance)	x/Ambulance
3	Final MGU	Both matched	θ={x/Ambulance}

(b) Prove Proceed(CarA) Using Resolution
Step 1: Relevant Knowledge
We only need Rules 1, 2, and 3.
Rule 1
Vehicle(x)∧EmergencyType(x)→ClearPath(x)
Rule 2
ClearPath(x)→GreenSignal(x)∧¬RedSignal(x)
Rule 3
Vehicle(x)∧Behind(x,y)∧GreenSignal(y)→Proceed(x)
Facts:
Vehicle(Ambulance) EmergencyType(Ambulance) Vehicle(CarA) Behind(CarA,Ambulance)

Step 2: Convert Rules into CNF
Rule 1
Original:
Vehicle(x)∧EmergencyType(x)→ClearPath(x)
Using:
A→B≡¬A∨B
we get:
¬Vehicle(x)∨¬EmergencyType(x)∨ClearPath(x)

Rule 2
Original:
ClearPath(x)→GreenSignal(x)∧¬RedSignal(x)
Because there are two conclusions, split it into two clauses:
Clause 2a:
¬ClearPath(x)∨GreenSignal(x)
Clause 2b:
¬ClearPath(x)∨¬RedSignal(x)
For proving Proceed(CarA), we only need Clause 2a.

Rule 3
Original:
Vehicle(x)∧Behind(x,y)∧GreenSignal(y)→Proceed(x)
CNF:
¬Vehicle(x)∨¬Behind(x,y)∨¬GreenSignal(y)∨Proceed(x)

Step 3: Use the Facts
We have:
Vehicle(Ambulance)
and
EmergencyType(Ambulance)
From Rule 1:
¬Vehicle(x)∨¬EmergencyType(x)∨ClearPath(x)
Substitute:
θ1={x/Ambulance}
We obtain:
¬Vehicle(Ambulance)∨¬EmergencyType(Ambulance)∨ClearPath(Ambulance)

Resolve with:
Vehicle(Ambulance)
to obtain:
¬EmergencyType(Ambulance)∨ClearPath(Ambulance)
Resolve with:
EmergencyType(Ambulance)
Therefore:
ClearPath(Ambulance)

Step 4: Derive GreenSignal(Ambulance)
From Rule 2a:
¬ClearPath(x)∨GreenSignal(x)
Substitute:
x=Ambulance
giving:
¬ClearPath(Ambulance)∨GreenSignal(Ambulance)
We already derived:
ClearPath(Ambulance)
Therefore, resolution gives:
GreenSignal(Ambulance)

Step 5: Apply Rule 3
Rule 3:
¬Vehicle(x)∨¬Behind(x,y)∨¬GreenSignal(y)∨Proceed(x)
We have:
Vehicle(CarA) Behind(CarA,Ambulance) GreenSignal(Ambulance)
Use the substitution:
θ2={x/CarA, y/Ambulance}
Rule 3 becomes:
¬Vehicle(CarA)∨¬Behind(CarA,Ambulance)∨¬GreenSignal(Ambulance)∨Proceed(CarA)
Resolve with the three known facts.
First:
Vehicle(CarA)
gives:
¬Behind(CarA,Ambulance)∨¬GreenSignal(Ambulance)∨Proceed(CarA)
Then:
Behind(CarA,Ambulance)
gives:
¬GreenSignal(Ambulance)∨Proceed(CarA)
Finally:
GreenSignal(Ambulance)
gives:
Proceed(CarA)

Resolution Proof Using Refutation
Another standard way to prove the goal is to assume its negation.
Goal:
Proceed(CarA)
Add the negated goal:
¬Proceed(CarA)
From Rule 3 and the facts, resolution derives:
Proceed(CarA)
Now we have both:
Proceed(CarA)
and
¬Proceed(CarA)

Resolving them produces the empty clause:
□
Therefore, the negated goal leads to a contradiction.
Hence:
Proceed(CarA) is true

(c) Handling Two Simultaneous Emergency Vehicles
Is the current knowledge base sufficient?

Partially, but not completely.
The existing FOL rules are universally quantified:
$\forall x$
Therefore, Rule 1 can independently handle multiple emergency vehicles.
For example, suppose we have:
Vehicle(Ambulance1) EmergencyType(Ambulance1)
and:
Vehicle(Ambulance2) EmergencyType(Ambulance2)
Rule 1 can derive:
ClearPath(Ambulance1)
and:
ClearPath(Ambulance2)
Similarly, Rule 2 can derive:
GreenSignal(Ambulance1)
and:
GreenSignal(Ambulance2)
So the basic logical representation supports multiple vehicles.

However, there is a problem
The current knowledge base does not specify how to handle conflicts between two emergency vehicles.
For example:
Ambulance 1 → needs Road A
Ambulance 2 → needs Road A
Both may receive:
GreenSignal
at the same time.
The knowledge base does not specify:

- Which emergency vehicle gets priority?
- What happens if both require the same road?
- How should conflicting green signals be handled?
- How should routes be selected?
- How should one emergency vehicle be delayed or rerouted?
- How should the system determine emergency priority?

Therefore, additional rules and facts are required.

Additional Rule 1: Emergency Priority
We can introduce:
$\forall x \forall y: EmergencyType(x) \land EmergencyType(y) \land Priority(x,y) \rightarrow GivePriority(x)$
This allows the system to identify which emergency vehicle should receive priority.

Additional Rule 2: Route Conflict
We can introduce:
$\forall x \forall y: EmergencyVehicle(x) \land EmergencyVehicle(y) \land SameRoute(x,y) \rightarrow RouteConflict(x,y)$
This identifies when two emergency vehicles need the same route.

Additional Rule 3: Resolve the Conflict
For example:
$\forall x \forall y: RouteConflict(x,y) \wedge HigherPriority(x,y) \rightarrow Delay(y)$
This means that if vehicle x has higher priority than vehicle y, vehicle y can be delayed.

Additional Facts Required
The system would also need facts such as:
EmergencyVehicle(Ambulance1) EmergencyVehicle(Ambulance2) PriorityLevel(Ambulance1,High) PriorityLevel(Ambulance2,Medium)
and possibly:
SameRoute(Ambulance1,Ambulance2)
These facts allow the agent to make a logical decision when two emergencies occur simultaneously.

Final Conclusion

Part	Answer
(a) Unification	MGU: $\theta=\{x/Ambulance\}$
(b) Resolution	ClearPath(Ambulance)→GreenSignal(Ambulance)→Proceed(CarA)
(c) Multiple emergencies	Basic handling is possible, but conflict resolution and priority rules are required

Key logical chain

$$Vehicle(Ambulance)+EmergencyType(Ambulance) \Downarrow ClearPath(Ambulance) \Downarrow GreenSignal(Ambulance) \Downarrow Vehicle(CarA)+Behind(CarA,Ambulance)+GreenSignal(Ambulance) \Downarrow Proceed(CarA)$$

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

ANSWER:

This case study demonstrates Propositional Logic, CNF conversion, Resolution Refutation, and logical reasoning.

(a) Convert P1, P2, P3 and Facts into CNF

Given Knowledge Base

P1:

SoilDry→IrrigationNeeded

P2:

IrrigationNeeded∧CropWheat→ApplyDripMethod

P3:

¬ApplyDripMethod∧CropWheat→CropAtRisk

Facts:

SoilDry=True CropWheat=True

Step 1: Convert P1 into CNF

Original:

SoilDry→IrrigationNeeded

Eliminate implication

We use:

A→B≡¬A∨B

Therefore:

¬SoilDry∨IrrigationNeeded

This is already in CNF.

P1 CNF:

¬SoilDry∨IrrigationNeeded

Step 2: Convert P2 into CNF

Original:

IrrigationNeeded∧CropWheat→ApplyDripMethod

Eliminate implication

Using:

A→B≡¬A∨B

where:

A=(IrrigationNeeded∧CropWheat)

we get:

¬(IrrigationNeeded∧CropWheat)∨ApplyDripMethod

Move ¬ inward
Using De Morgan's law:
$\neg(A \land B) \equiv \neg A \lor \neg B$
Therefore:
$(\neg IrrigationNeeded \lor \neg CropWheat) \lor ApplyDripMethod$
Remove unnecessary brackets:
$\neg IrrigationNeeded \lor \neg CropWheat \lor ApplyDripMethod$
This is already in CNF.

Step 3: Convert P3 into CNF
Original:
$\neg ApplyDripMethod \land CropWheat \rightarrow CropAtRisk$
Eliminate implication
$\neg(\neg ApplyDripMethod \land CropWheat) \lor CropAtRisk$
Move ¬ inward
Using De Morgan's law:
$\neg(A \land B) = \neg A \lor \neg B$
Therefore:
$\neg(\neg ApplyDripMethod) \lor \neg CropWheat \lor CropAtRisk$
Double negation gives:
$ApplyDripMethod \lor \neg CropWheat \lor CropAtRisk$
P3 CNF:
$ApplyDripMethod \lor \neg CropWheat \lor CropAtRisk$

Step 4: Convert the Facts into CNF
The facts are already atomic propositions.
Fact 1:
SoilDry
Fact 2:
CropWheat
Both are valid CNF clauses.

Final CNF Knowledge Base

Sentence	CNF
P1	$\neg SoilDry \lor IrrigationNeeded$
P2	$\neg IrrigationNeeded \lor \neg CropWheat \lor ApplyDripMethod$
P3	$ApplyDripMethod \lor \neg CropWheat \lor CropAtRisk$
Fact 1	SoilDry
Fact 2	CropWheat

Important
No distribution step is actually required here because after implication elimination and moving negations inward, each expression is already a single disjunction (a CNF clause).

(b) Resolution Refutation to Prove ApplyDripMethod
Goal
We want to prove:
ApplyDripMethod
Resolution Refutation works by:

1. Negating the goal.
2. Adding the negated goal to the KB.
3. Applying resolution.
4. Deriving the empty clause □.

Step 1: Negate the Goal
Goal:
ApplyDripMethod
Negated goal:
¬ApplyDripMethod
Add this to the CNF knowledge base.

Step 2: List the Relevant Clauses
Clause C1 — P1
¬SoilDry∨IrrigationNeeded
Clause C2 — P2
¬IrrigationNeeded∨¬CropWheat∨ApplyDripMethod
Clause C3 — P3
ApplyDripMethod∨¬CropWheat∨CropAtRisk
Clause C4 — Fact
SoilDry
Clause C5 — Fact
CropWheat
Clause C6 — Negated goal
¬ApplyDripMethod
For proving ApplyDripMethod, C3 is not needed. C1, C2, C4, C5 and C6 are sufficient.

Step 3: Resolution Derivation
Resolution 1
Take C1:
¬SoilDry∨IrrigationNeeded
and C4:
SoilDry
The complementary literals are:
¬SoilDry
and
SoilDry
Resolving them gives:
IrrigationNeeded
Call this C7.

Resolution 2
Take C2:
¬IrrigationNeeded∨¬CropWheat∨ApplyDripMethod
and C7:
IrrigationNeeded
Resolve:
¬IrrigationNeeded
with:
IrrigationNeeded
Result:
¬CropWheat∨ApplyDripMethod
Call this C8.

Resolution 3
Take C8:
¬CropWheat∨ApplyDripMethod
and C5:
CropWheat
Resolve:

¬CropWheat
with:
CropWheat
Therefore:
ApplyDripMethod
Call this C9.

Resolution 4 — Derive Empty Clause
We have:
C9:
ApplyDripMethod
C6:
¬ApplyDripMethod
Resolve:
ApplyDripMethod
with:
¬ApplyDripMethod
There are no remaining literals.
Therefore:
□
The empty clause has been derived.
Hence:
∴ApplyDripMethod
is logically proven.

Resolution Proof Tree
You can draw the proof tree in your exam like this:

```
                    □
                   / \
                  /   \
      ApplyDripMethod    ¬ApplyDripMethod
          /               \
         /                 \
 ¬CropWheat ∨ ApplyDripMethod     Negated Goal
       /       \
      /         \
  CropWheat      IrrigationNeeded
     |             |
   Fact C5             |
                   |
          ¬SoilDry ∨ IrrigationNeeded
                   |
                SoilDry
                 Fact C4
```

A cleaner step-by-step resolution tree is:
C1: ¬SoilDry ∨ IrrigationNeeded
+
C4: SoilDry
|
▼
C7: IrrigationNeeded
+
C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
|
▼
C8: ¬CropWheat ∨ ApplyDripMethod
+
C5: CropWheat
|
▼
C9: ApplyDripMethod
+
C6: ¬ApplyDripMethod
|
▼
□
Therefore:
ApplyDripMethod is true

(c) Remove the Fact SoilDry
Now suppose:
SoilDry
is removed from the knowledge base.
The remaining facts are:
CropWheat
The rules remain:
SoilDry→IrrigationNeeded IrrigationNeeded∧CropWheat→ApplyDripMethod
¬ApplyDripMethod∧CropWheat→CropAtRisk

Step 1: Can we derive IrrigationNeeded?
P1 says:
SoilDry→IrrigationNeeded
But we no longer have:
SoilDry
Therefore, we cannot derive:
IrrigationNeeded
So:
IrrigationNeeded cannot be concluded

Step 2: Can we derive ApplyDripMethod?
P2 requires:
IrrigationNeeded∧CropWheat
We know:
CropWheat
but we do not know:
IrrigationNeeded
Therefore, P2 cannot fire.
Thus:
ApplyDripMethod cannot be concluded

Step 3: Can we derive CropAtRisk?
P3 requires:
¬ApplyDripMethod∧CropWheat
We know:
CropWheat
However, we do not know:
¬ApplyDripMethod
Not being able to prove ApplyDripMethod does not automatically mean ¬ApplyDripMethod.
Therefore, P3 cannot fire.
Hence:
CropAtRisk cannot be concluded

Important Logical Point
This is a very important concept for the exam:
Failure to prove P ≠ proof of ¬P
In other words:
We cannot conclude that the drip method is not applied merely because we cannot prove that it is applied.
So:
¬ApplyDripMethod
is not available as a fact.
Therefore, we cannot derive:
CropAtRisk

Comparison Before and After Removing SoilDry

Conclusion	SoilDry present	SoilDry removed
SoilDry	True	Unknown
CropWheat	True	True
IrrigationNeeded	Derived	Not derived
ApplyDripMethod	Derived	Not derived
¬ApplyDripMethod	Not given	Not given
CropAtRisk	Not derived	Not derived

Hence:
CropAtRisk does not follow
Reason: absence of proof for ApplyDripMethod is not proof of its negation.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

ANSWER:
This case study demonstrates Propositional Logic, Knowledge Representation, Modus Ponens, Forward Chaining, and logical decision-making.

(a) Construct the Agent's Belief State
Given Percepts
The agent perceives:

1. Stench at [1,2]
2. Breeze at [1,1]
3. Glitter at [2,2]

We represent them as:
Stench[1,2] Breeze[1,1] Glitter[2,2]

Step 1: Apply Rule 1 — Stench
Rule:
Stench[1,2]→WumpusAdjacent[1,2]
Fact:
Stench[1,2]
By Modus Ponens:
WumpusAdjacent[1,2]
This means the Wumpus is in one of the cells adjacent to [1,2].
The adjacent cells are:
[1,1],[1,3],[2,2]
Therefore:
Wumpus∈{[1,1],[1,3],[2,2]}
assuming these cells exist in the world.

Step 2: Apply Rule 2 — Breeze
Rule:
Breeze[1,1]→PitAdjacent[1,1]
Breeze[1,1]
By Modus Ponens:
PitAdjacent[1,1]
The cells adjacent to [1,1] are:
[1,2],[2,1]
Therefore:
Pit∈{[1,2],[2,1]}

Step 3: Apply Rule 3 — Glitter
Rule:
Glitter[x,y]→Gold[x,y]
Given:
Glitter[2,2]
Substitute:
x=2,y=2
Therefore:
Gold[2,2]
So the agent knows that the gold is at [2,2].

Step 4: Apply Rule 4 — Safe Cell
Rule:
¬Wumpus[x,y]∧¬Pit[x,y]→Safe[x,y]
To apply this rule, the agent must know both:
¬Wumpus[x,y]
and
¬Pit[x,y]
However, the given percepts do not provide enough information to establish these negative facts for the relevant cells.
Therefore, Rule 4 cannot currently be applied.

Belief State
After applying Modus Ponens, the agent knows:

Percept / Conclusion	Knowledge
Stench [1,2]	Wumpus is adjacent to [1,2]
Breeze [1,1]	Pit is adjacent to [1,1]
Glitter [2,2]	Gold is at [2,2]
Wumpus candidates	[1,1], [1,3], [2,2]
Pit candidates	[1,2], [2,1]
Gold location	[2,2]
Safe cells	Cannot be established from given rules alone

Important:
A Stench does not mean the Wumpus is in the same cell. It means the Wumpus is in an adjacent cell.
Similarly, a Breeze does not mean there is a pit in [1,1]. It means a pit is adjacent to [1,1].

(b) Forward Chaining to Identify Possible Wumpus and Pit Locations
Step 1: Start with the percepts
Initial facts:
Stench[1,2] Breeze[1,1] Glitter[2,2]

Step 2: Forward Chain the Stench
Rule 1:
Stench[1,2]→WumpusAdjacent[1,2]
Since:
Stench[1,2]=True
derive:
WumpusAdjacent[1,2]
Adjacent cells:
[1,1],[1,3],[2,2]
Therefore:
Wumpus∈{[1,1],[1,3],[2,2]}

Step 3: Forward Chain the Breeze
Rule 2:
Breeze[1,1]→PitAdjacent[1,1]
Since:
Breeze[1,1]=True
derive:
PitAdjacent[1,1]
Adjacent cells:
[1,2],[2,1]
Therefore:
Pit∈{[1,2],[2,1]}

Step 4: Forward Chain the Glitter
Rule 3:
Glitter[x,y]→Gold[x,y]
Given:
Glitter[2,2]
derive:
Gold[2,2]

Forward Chaining Summary

```
Stench[1,2]
    |
    | Rule 1
    ↓
Wumpus adjacent to [1,2]
    |
    ↓
Possible Wumpus:
[1,1], [1,3], [2,2]


Breeze[1,1]
    |
    | Rule 2
    ↓
Pit adjacent to [1,1]
    |
    ↓
Possible Pit:
[1,2], [2,1]


Glitter[2,2]
    |
    | Rule 3
    ↓
Gold[2,2]
```

Final possible locations
Wumpus={[1,1],[1,3],[2,2]} Pit={[1,2],[2,1]} Gold=[2,2]

(c) Is the Path [1,1] → [1,2] → [2,2] Safe?
The proposed path is:
[1,1]→[1,2]→[2,2]
We need to check every cell.

Step 1: Check [1,1]
From the stench information:
$Wumpus \in \{[1,1],[1,3],[2,2]\}$
Therefore, [1,1] is a possible Wumpus location.
The agent does not have:
$\neg Wumpus[1,1]$
Also, there is no evidence that [1,1] is definitely free from a pit.
Therefore:
[1,1] is not proven safe

Step 2: Check [1,2]
From the breeze:
$Pit \in \{[1,2],[2,1]\}$
Therefore:
[1,2]
is a possible pit location.
Also, the stench at [1,2] means a Wumpus is adjacent to [1,2], but does not itself mean there is a Wumpus in [1,2].
However, because a pit at [1,2] has not been ruled out:
[1,2] is not proven safe

Step 3: Check [2,2]
We know:
$Gold[2,2]$
However, from the stench at [1,2]:
$Wumpus \in \{[1,1],[1,3],[2,2]\}$
Therefore, [2,2] is also a possible Wumpus location.
Hence:
[2,2] is not proven safe

Safety Analysis

Cell	Possible Danger	Safe?
[1,1]	Possible Wumpus	Not proven safe
[1,2]	Possible Pit	Not proven safe
[2,2]	Possible Wumpus	Not proven safe
[2,2]	Gold present	Gold location known

Therefore, the complete path:
$[1,1] \rightarrow [1,2] \rightarrow [2,2]$
cannot be logically guaranteed as safe using the given knowledge.
The path is NOT proven safe.

Why Can't Rule 4 Prove Safety?
Rule 4 is:
$\neg Wumpus[x,y] \wedge \neg Pit[x,y] \rightarrow Safe[x,y]$
For example, to prove:
$Safe[2,2]$
we need:
$\neg Wumpus[2,2]$
$\neg Pit[2,2]$
But we have neither of these negative facts.
In fact, [2,2] is a possible Wumpus location, so we cannot conclude:
$\neg Wumpus[2,2]$
Therefore, Rule 4 cannot establish safety.

Final Exam Answer

(a) Belief State

Using Modus Ponens:

$Stench[1,2] \rightarrow WumpusAdjacent[1,2]$

Therefore:

$Wumpus \in \{[1,1],[1,3],[2,2]\}$

Similarly:

$Breeze[1,1] \rightarrow PitAdjacent[1,1]$

Therefore:

$Pit \in \{[1,2],[2,1]\}$

And:

$Glitter[2,2] \rightarrow Gold[2,2]$

Therefore:

$Gold[2,2]$

(b) Forward Chaining

$Stench[1,2] \Rightarrow WumpusAdjacent[1,2] \Rightarrow Wumpus \in \{[1,1],[1,3],[2,2]\}$
$Breeze[1,1] \Rightarrow PitAdjacent[1,1] \Rightarrow Pit \in \{[1,2],[2,1]\}$ $Glitter[2,2] \Rightarrow Gold[2,2]$

(c) Path Safety

The path is:

$[1,1] \rightarrow [1,2] \rightarrow [2,2]$

- [1,1] may contain the Wumpus.
- [1,2] may contain a Pit.
- [2,2] may contain the Wumpus.
- [2,2] contains the Gold.

Since the agent cannot establish both ¬Wumpus and ¬Pit for every cell, Rule 4 cannot prove the cells safe.
∴ [1,1]→[1,2]→[2,2] is NOT guaranteed safe.
Key point: In Wumpus World, unknown is not the same as safe. The agent should only classify a cell as definitely safe when the KB logically establishes that there is neither a Wumpus nor a pit.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand